SYNAPSE Testing Strategy

Engineering Team

March 7, 2026

Contents

1 Introduction and Testing Philosophy **2**
1.1 Testing Goals . . . 2
1.2 Testing Pyramid . . . 2
1.3 Test-Driven Development Approach . . . 3
1.4 Continuous Testing in CI/CD Pipeline . . . 3
1.5 Definition of Done . . . 3
1.6 Code Coverage Targets . . . 4
1.7 Testing Environments . . . 4

2 Testing Stack **5**
2.1 Backend Testing Tools . . . 5
2.1.1 pytest 7.x and pytest-django . . . 5
2.1.2 pytest-asyncio . . . 5
2.1.3 pytest-cov . . . 6
2.1.4 factory_boy and faker . . . 6
2.1.5 httpx and responses . . . 7
2.1.6 freezegun . . . 7
2.2 Frontend Testing Tools . . . 8
2.2.1 Jest 29.x and React Testing Library . . . 8
2.2.2 Mock Service Worker (MSW) . . . 8
2.2.3 Playwright . . . 9
2.3 Load Testing: Locust . . . 9
2.4 Security Testing Tools . . . 10
2.4.1 bandit . . . 10
2.4.2 OWASP ZAP . . . 11
2.4.3 safety . . . 11

3 Unit Testing **12**
3.1 Backend Unit Tests . . . 12
3.1.1 Django Model Tests . . . 12
3.1.2 Django Serializer Tests . . . 13
3.1.3 Service and Utility Function Tests . . . 14
3.1.4 NLP Pipeline Unit Tests . . . 15
3.1.5 Celery Task Unit Tests . . . 16
3.1.6 Agent Tool Unit Tests . . . 17
3.2 Frontend Unit Tests . . . 18
3.2.1 React Component Tests . . . 18
3.2.2 Custom Hook Tests . . . 20
3.2.3 Utility Function Tests . . . 21

4 Integration Testing **23**
4.1 API Integration Tests . . . 23
4.1.1 Authentication Endpoints . . . 23
4.1.2 Articles Endpoints . . . 24
4.2 Database Integration Tests . . . 25
4.3 Cache Integration Tests . . . 26
4.4 Celery Integration Tests . . . 27

5 End-to-End Testing with Playwright **28**
5.1 Authentication Flows . . . 28
5.2 Dashboard and Article Flows . . . 29
5.3 AI Chat Flow . . . 30
5.4 Workflow Automation E2E . . . 30

6 Performance Testing with Locust **32**
6.1 Load Test Scenarios . . . 32
6.1.1 Scenario 1: Normal Load . . . 32
6.1.2 Scenario 2: Peak Load and Stress Test . . . 33
6.2 Performance Targets . . . 33
6.3 Frontend Performance Testing . . . 34
6.4 Database Query Optimization . . . 34

7 Security Testing **35**
7.1 Static Analysis with bandit . . . 35
7.2 Dynamic Security Scanning with OWASP ZAP . . . 35
7.3 Dependency Vulnerability Scanning . . . 35
7.4 Manual Security Tests . . . 36

8 AI and ML Testing **37**
8.1 Summarization Quality Tests . . . 37
8.2 Topic Classification Tests . . . 37
8.3 Semantic Search Relevance Tests . . . 38
8.4 Embedding Consistency Tests . . . 38
8.5 RAG Hallucination Detection . . . 39

9 Test Data Management **41**
9.1 Factory Definitions . . . 41
9.2 Test Database Isolation . . . 42
9.3 Mock Strategies for External APIs . . . 42

10 CI/CD Test Integration **44**
10.1 GitHub Actions Test Pipeline . . . 44
10.2 Test Parallelization . . . 46
10.3 Test Failure Notifications . . . 46

11 Test Environment Configuration **48**
11.1 pytest Configuration . . . 48
11.2 conftest.py Structure . . . 48
11.3 Jest Configuration . . . 49
11.4 Docker Compose Test Override . . . 50

12 Quality Metrics and Reporting **52**
12.1 Coverage Targets by Module . . . 52
12.2 Test Execution Time Budgets . . . 52
12.3 Quality Gates . . . 52
12.4 Bug Escape Rate Tracking . . . 53
12.5 Test Health Dashboard . . . 53

13 Accessibility and Compliance Testing **54**
13.1 Automated Accessibility Testing . . . 54
13.2 Keyboard Navigation E2E Tests . . . 55
13.3 Screen Reader Testing . . . 55

14 Test Maintenance and Evolution **57**
14.1 Flaky Test Management . . . 57
14.2 Test Code Quality . . . 57
14.3 Regression Test Suite . . . 57

15 Continuous Testing Culture **59**
15.1 Testing Best Practices . . . 59
15.2 Team Training and Documentation . . . 59
15.3 Testing Tools and Libraries Summary . . . 59

16 Conclusion **61**

Chapter 1

Introduction and Testing Philosophy

1.1 Testing Goals

The SYNAPSE platform is a FAANG-style AI-powered technology intelligence platform that demands the highest standards of reliability, correctness, performance, and security. Our testing strategy is designed to achieve the following overarching goals:

1. **Reliability**: Every deployment to production must be verifiable and stable. Critical functionality must never degrade between releases.
2. **Correctness**: Business logic, data integrity, and algorithmic outputs must be validated against expected behaviors through automated tests.
3. **Performance**: Response times, throughput, and resource utilization must meet defined service-level objectives.
4. **Security**: Authentication, authorization, data validation, and protection against common vulnerabilities must be continuously verified.

The SYNAPSE platform is built on a modern polyglot stack: Django 4.2 and FastAPI for the backend, Next.js 14 for the frontend, PostgreSQL 15 with Redis 7 for data persistence and caching, Celery for asynchronous task processing, and LangChain for AI-powered features.

1.2 Testing Pyramid

We follow the industry-standard testing pyramid, ensuring a distribution of test types that balances speed, coverage, and confidence:

```
                /\
               /  \
              / E2E \        End-to-End Tests (10%)
             /------\       Playwright-based browser tests
            /        \
           /          \
          /Integration \    Integration Tests (30%)
         /Tests (DB,    \   API contracts, workflows
        /Workflows,      \
       /API calls)        \
      /                    \
     /                      \
    /                        \
```

```
  /--------Unit Tests------\  Unit Tests (60%)
 / (Django, FastAPI, React)  \ Fastest, most isolated
/____________________________\
```

Unit Tests (60% of test suite): Fast, isolated tests of individual functions, classes, and components. These run on every commit and should complete in under 2 minutes.

Integration Tests (30% of test suite): Tests that verify interactions between components, including API contracts, database transactions, cache behavior, and Celery task workflows. These should complete in under 10 minutes.

End-to-End Tests (10% of test suite): Full browser-based tests using Playwright that simulate real user workflows. These are slower but provide the highest confidence that the product works as intended. These should complete in under 30 minutes.

1.3 Test-Driven Development Approach

For critical business logic and bug fixes, we adopt a Test-Driven Development (TDD) approach:

1. Write failing tests first that specify the desired behavior
2. Implement the minimum code to make tests pass
3. Refactor for clarity and maintainability
4. Ensure tests remain passing and coverage improves

This approach is mandatory for:

- Authentication and authorization logic
- Data validation and serialization
- Workflow execution and scheduling
- NLP pipeline processing
- Agent tool implementations

1.4 Continuous Testing in CI/CD Pipeline

Every change to the codebase is automatically tested through our GitHub Actions CI/CD pipeline:

- **On every PR**: Code style checks, linting, unit tests, and integration tests run automatically
- **On merge to main**: Full test suite including security scans and build verification
- **Nightly scheduled runs**: End-to-end tests, performance tests, and comprehensive security scans

1.5 Definition of Done

Code is considered complete and ready for production merge only when:

1. All unit tests pass with no failures
2. All integration tests pass with no failures
3. Code coverage meets or exceeds minimum thresholds (see Section 1.6)
4. No new critical or high-severity security vulnerabilities are introduced

5. Code review approval obtained from at least one team member
6. CI pipeline reports PASS on all checks
7. No flaky tests or timeouts introduced

1.6 Code Coverage Targets

- **Backend (Django + FastAPI)**: Minimum 80% code coverage. Critical paths (auth, data models, NLP) require 95%+ coverage.
- **Frontend (React/Next.js)**: Minimum 70% code coverage. Components that handle data transformation or state logic require 85%+ coverage.
- **Excluded from coverage**: Generated code, migrations, configuration files, and trivial utility functions.

Coverage is measured using pytest-cov for backend and Jest coverage reports for frontend. Coverage enforcement is automated in the CI pipeline: pull requests that decrease coverage are automatically flagged for review.

1.7 Testing Environments

- **Development**: Local machine, isolated test database, mocked external APIs. Used for rapid iteration and debugging.
- **Staging**: Production-like environment with real external API credentials (sandbox mode), realistic data volumes, and identical infrastructure to production. Used for final validation before release.
- **Production**: Live environment with real users and data. No automated tests should directly run against production; instead, synthetic monitoring and canary deployments are used.

Test data in staging is refreshed daily from sanitized production snapshots with all PII removed.

Chapter 2

Testing Stack

2.1 Backend Testing Tools

The SYNAPSE backend relies on the following testing libraries and frameworks:

2.1.1 pytest 7.x and pytest-django

pytest is the primary test runner for all Python code. pytest-django provides Django-specific fixtures and database management.

```python
import pytest
from django.test import TransactionTestCase
from articles.models import Article
from articles.factories import ArticleFactory

@pytest.mark.django_db
class TestArticleModel:
    def test_article_creation(self):
        article = ArticleFactory(
            title="Breaking AI News",
            url="https://example.com/ai-news"
        )
        assert article.title == "Breaking AI News"
        assert Article.objects.count() == 1

    def test_article_slug_generation(self):
        article = ArticleFactory(title="Machine Learning Advances")
        assert article.slug == "machine-learning-advances"

    def test_article_duplicate_url_constraint(self):
        url = "https://example.com/unique-article"
        ArticleFactory(url=url)
        with pytest.raises(Exception):  # IntegrityError
            ArticleFactory(url=url)
```

Listing 2.1: Basic pytest usage in SYNAPSE

2.1.2 pytest-asyncio

For testing FastAPI endpoints and async functions, pytest-asyncio is used:

```python
import pytest
from httpx import AsyncClient
```

```python
from main import app

@pytest.mark.asyncio
async def test_semantic_search_endpoint():
    async with AsyncClient(app=app, base_url="http://test") as client:
        response = await client.post(
            "/api/v1/search/semantic",
            json={"query": "artificial intelligence trends"}
        )
        assert response.status_code == 200
        data = response.json()
        assert "results" in data
        assert len(data["results"]) > 0
```

Listing 2.2: Testing async FastAPI endpoints

2.1.3 pytest-cov

Code coverage is measured using pytest-cov, configured in pyproject.toml to exclude test files and migrations:

```toml
[tool.pytest.ini_options]
addopts = --cov=apps --cov-report=html --cov-report=term-missing
testpaths = ["tests"]
python_files = ["test_*.py"]
DJANGO_SETTINGS_MODULE = "config.settings.test"
```

Listing 2.3: pytest-cov configuration in pyproject.toml

2.1.4 factory_boy and faker

Test data factories ensure consistent, realistic test data:

```python
import factory
from faker import Faker
from django.contrib.auth import get_user_model
from articles.models import Article, Topic

fake = Faker()
User = get_user_model()

class UserFactory(factory.django.DjangoModelFactory):
    class Meta:
        model = User

    username = factory.Sequence(lambda n: f"user_{n}")
    email = factory.LazyAttribute(lambda o: fake.email())
    password = factory.PostGenerationMethodCall('set_password', 'SecurePassword123!')
    is_active = True

class TopicFactory(factory.django.DjangoModelFactory):
    class Meta:
        model = Topic

    name = factory.Faker('word')
    description = factory.Faker('sentence')

class ArticleFactory(factory.django.DjangoModelFactory):
    class Meta:
```

```
        model = Article

    title = factory.Faker('sentence', nb_words=6)
    url = factory.LazyAttribute(lambda o: fake.url())
    summary = factory.Faker('paragraph')
    source = factory.LazyAttribute(lambda o: fake.domain_name())
    topic = factory.SubFactory(TopicFactory)
    published_at = factory.Faker('date_time_this_year')
```

Listing 2.4: Factory definitions using factory_boy

2.1.5 httpx and responses

For testing async HTTP requests and mocking external APIs:

```
import pytest
import responses
from httpx import AsyncClient
from integrations.scrapers import scrape_hackernews

@responses.activate
async def test_scrape_hackernews_mocked():
    responses.add(
        responses.GET,
        "https://news.ycombinator.com/",
        json={"stories": [{"id": 1, "title": "AI Breakthrough"}]},
        status=200
    )

    stories = await scrape_hackernews()
    assert len(stories) == 1
    assert stories[0]["title"] == "AI Breakthrough"
```

Listing 2.5: Mocking external HTTP calls with responses

2.1.6 freezegun

Testing time-dependent logic by mocking datetime:

```
import pytest
from freezegun import freeze_time
from datetime import datetime
from workflows.models import Workflow

@freeze_time("2024-01-15 10:00:00")
def test_workflow_scheduled_execution():
    workflow = WorkflowFactory(
        cron_expression="0 9 * * *",  # 9 AM daily
        next_run_at=datetime(2024, 1, 16, 9, 0, 0)
    )
    assert workflow.should_run_now() == False

    # Move time to next scheduled run
    with freeze_time("2024-01-16 09:00:00"):
        assert workflow.should_run_now() == True
```

Listing 2.6: Mocking time with freezegun

2.2 Frontend Testing Tools

2.2.1 Jest 29.x and React Testing Library

Jest is the test runner for all JavaScript/TypeScript code. React Testing Library encourages testing from the user's perspective:

```
import { render, screen, fireEvent } from '@testing-library/react';
import ArticleCard from '@/components/ArticleCard';

describe('ArticleCard Component', () => {
  const mockArticle = {
    id: '1',
    title: 'AI Trends 2024',
    summary: 'Latest developments...',
    published_at: '2024-01-15',
    source: 'TechNews',
    topic: 'AI'
  };

  test('renders article with all props', () => {
    render(<ArticleCard article={mockArticle} />);
    expect(screen.getByText('AI Trends 2024')).toBeInTheDocument();
    expect(screen.getByText('TechNews')).toBeInTheDocument();
  });

  test('bookmark button toggles state', () => {
    const mockOnBookmark = jest.fn();
    render(
      <ArticleCard
        article={mockArticle}
        onBookmark={mockOnBookmark}
      />
    );

    const bookmarkBtn = screen.getByRole('button', { name: /bookmark/i });
    fireEvent.click(bookmarkBtn);
    expect(mockOnBookmark).toHaveBeenCalledWith('1');
  });
});
```

Listing 2.7: Component test with React Testing Library

2.2.2 Mock Service Worker (MSW)

API mocking at the network level allows realistic testing without external API calls:

```
import { setupServer } from 'msw/node';
import { rest } from 'msw';

export const server = setupServer(
  rest.get('/api/v1/articles', (req, res, ctx) => {
    return res(
      ctx.status(200),
      ctx.json({
        results: [
          {
            id: '1',
            title: 'Article 1',
```

```
            summary: 'Summary 1'
          }
        ],
        total: 1
      })
    );
  }),

  rest.post('/api/v1/ai/chat', (req, res, ctx) => {
    return res(
      ctx.status(200),
      ctx.json({
        message_id: 'msg-123',
        content: 'This is an AI response',
        sources: [{ article_id: '1', relevance: 0.95 }]
      })
    );
  })
);

beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());
```

Listing 2.8: MSW setup for API mocking

2.2.3 Playwright

End-to-end browser automation for full workflow testing:

```
import { test, expect } from '@playwright/test';

test.describe('Authentication Flow', () => {
  test('register, login, and logout', async ({ page }) => {
    await page.goto('http://localhost:3000/register');

    await page.fill('input[name="email"]', 'newuser@test.com');
    await page.fill('input[name="password"]', 'SecurePass123!');
    await page.fill('input[name="confirmPassword"]', 'SecurePass123!');

    await page.click('button[type="submit"]');
    await expect(page).toHaveURL(/.*\/dashboard/);

    const userMenu = page.locator('[data-testid="user-menu"]');
    await userMenu.click();
    await page.click('button:has-text("Logout")');

    await expect(page).toHaveURL(/.*\/login/);
  });
});
```

Listing 2.9: Playwright E2E test example

2.3 Load Testing: Locust

Locust provides Python-based load testing with realistic user workflows:

```python
from locust import HttpUser, task, between
from random import randint

class SynapseUser(HttpUser):
    wait_time = between(1, 3)

    def on_start(self):
        # Login before starting tasks
        self.client.post(
            "/api/v1/auth/login",
            json={"email": "test@example.com", "password": "password"}
        )

    @task(6)
    def browse_dashboard(self):
        # 60% of traffic
        self.client.get(
            f"/api/v1/articles/?page=1&page_size=20",
            name="/api/v1/articles/"
        )

    @task(2)
    def semantic_search(self):
        # 20% of traffic
        self.client.post(
            "/api/v1/search/semantic",
            json={"query": "artificial intelligence trends"}
        )

    @task(1)
    def chat(self):
        # 10% of traffic
        self.client.post(
            "/api/v1/ai/chat",
            json={"message": "What are the latest AI breakthroughs?"}
        )

    @task(1)
    def bookmark_article(self):
        # 10% of traffic
        article_id = randint(1, 1000)
        self.client.post(f"/api/v1/articles/{article_id}/bookmark")
```

Listing 2.10: Locust load test scenarios

2.4 Security Testing Tools

2.4.1 bandit

Static analysis for Python security issues:

```bash
bandit -r apps/ -f json -o bandit-report.json
```

Listing 2.11: Running bandit security scan

2.4.2 OWASP ZAP

Dynamic security scanning for web vulnerabilities:

```
docker run -t owasp/zap2docker-stable zap-baseline.py \
  -t http://localhost:8000 \
  -r zap-report.html
```

Listing 2.12: OWASP ZAP baseline scan

2.4.3 safety

Checks Python dependencies for known vulnerabilities:

```
pip install safety
safety check --json > safety-report.json
```

Listing 2.13: Running safety check

Chapter 3

Unit Testing

3.1 Backend Unit Tests

3.1.1 Django Model Tests

Model tests verify data constraints, validations, and string representations:

```python
import pytest
from django.core.exceptions import ValidationError
from articles.models import Article, User
from articles.factories import ArticleFactory, UserFactory

@pytest.mark.django_db
class TestArticleModel:
    def test_article_creation_with_valid_data(self):
        article = ArticleFactory(
            title="Machine Learning in Healthcare",
            url="https://example.com/ml-healthcare",
            source="MedTech Weekly"
        )
        assert article.title == "Machine Learning in Healthcare"
        assert article.source == "MedTech Weekly"
        assert str(article) == "Machine Learning in Healthcare"

    def test_article_slug_generation_from_title(self):
        article = ArticleFactory(title="Quantum Computing Breakthroughs")
        assert article.slug == "quantum-computing-breakthroughs"

    def test_article_duplicate_url_unique_constraint(self):
        url = "https://example.com/unique-url"
        ArticleFactory(url=url)
        with pytest.raises(Exception):  # IntegrityError
            ArticleFactory(url=url)

    def test_article_published_at_timestamp(self):
        article = ArticleFactory()
        assert article.published_at is not None
        assert article.created_at is not None

@pytest.mark.django_db
class TestUserModel:
    def test_user_creation_with_valid_data(self):
        user = UserFactory(
            username="testuser",
```

```
            email="test@example.com"
        )
        assert user.username == "testuser"
        assert user.is_active == True

    def test_user_invalid_email_validation(self):
        with pytest.raises(ValidationError):
            user = UserFactory(email="invalid-email")
            user.full_clean()

    def test_user_weak_password_validation(self):
        with pytest.raises(Exception):
            user = UserFactory()
            user.set_password("123")  # Too weak

    def test_user_role_permissions(self):
        admin = UserFactory(is_staff=True, is_superuser=True)
        regular = UserFactory(is_staff=False)
        assert admin.has_perm("articles.change_article")
        assert not regular.has_perm("articles.delete_article")

@pytest.mark.django_db
class TestWorkflowModel:
    def test_workflow_cron_validation(self):
        valid_cron = "0 9 * * *"  # 9 AM daily
        workflow = WorkflowFactory(cron_expression=valid_cron)
        assert workflow.cron_expression == valid_cron

        invalid_cron = "invalid cron"
        with pytest.raises(ValidationError):
            workflow = WorkflowFactory(cron_expression=invalid_cron)
            workflow.full_clean()
```

Listing 3.1: Django model unit tests

3.1.2 Django Serializer Tests

Serializers handle data validation and transformation:

```
import pytest
from rest_framework.exceptions import ValidationError
from articles.serializers import ArticleSerializer, UserRegistrationSerializer
from articles.factories import ArticleFactory, UserFactory

class TestArticleSerializer:
    def test_article_serializer_valid_data(self):
        article_data = {
            "title": "New AI Research",
            "url": "https://example.com/ai-research",
            "summary": "Summary of research",
            "source": "Research Lab"
        }
        serializer = ArticleSerializer(data=article_data)
        assert serializer.is_valid()
        assert serializer.errors == {}

    def test_article_serializer_missing_required_fields(self):
        article_data = {
            "title": "Missing URL"
```

```
            # url is required but missing
        }
        serializer = ArticleSerializer(data=article_data)
        assert not serializer.is_valid()
        assert "url" in serializer.errors

    def test_article_serializer_invalid_url_format(self):
        article_data = {
            "title": "Bad URL",
            "url": "not-a-valid-url",
            "summary": "Test"
        }
        serializer = ArticleSerializer(data=article_data)
        assert not serializer.is_valid()

class TestUserRegistrationSerializer:
    def test_registration_valid_data(self):
        data = {
            "username": "newuser",
            "email": "new@example.com",
            "password": "SecurePassword123!"
        }
        serializer = UserRegistrationSerializer(data=data)
        assert serializer.is_valid()

    def test_registration_duplicate_email(self):
        UserFactory(email="existing@example.com")
        data = {
            "username": "newuser",
            "email": "existing@example.com",
            "password": "SecurePassword123!"
        }
        serializer = UserRegistrationSerializer(data=data)
        assert not serializer.is_valid()
        assert "email" in serializer.errors
```

Listing 3.2: Serializer validation tests

3.1.3 Service and Utility Function Tests

```
import pytest
from utils.text_processing import clean_text, extract_keywords
from utils.similarity import calculate_similarity_score
from utils.parsing import parse_cron_expression
from utils.files import generate_document_filename

class TestTextProcessing:
    def test_text_cleaning_removes_html_tags(self):
        dirty = "<p>Hello <b>World</b></p>"
        clean = clean_text(dirty)
        assert clean == "Hello World"

    def test_text_cleaning_normalizes_whitespace(self):
        messy = "Text  with   irregular   spacing"
        clean = clean_text(messy)
        assert clean == "Text with irregular spacing"

class TestKeywordExtraction:
```

```python
    def test_keyword_extraction_returns_list(self):
        text = "Machine learning and deep learning are AI subfields"
        keywords = extract_keywords(text)
        assert isinstance(keywords, list)
        assert len(keywords) > 0

    def test_keyword_extraction_case_insensitive(self):
        text = "Python PYTHON python"
        keywords = extract_keywords(text)
        # Should deduplicate case-insensitively
        python_count = sum(1 for k in keywords if k.lower() == "python")
        assert python_count == 1

class TestSimilarityScore:
    def test_similarity_score_identical_texts(self):
        text = "Machine learning is powerful"
        score = calculate_similarity_score(text, text)
        assert score == 1.0

    def test_similarity_score_completely_different(self):
        score = calculate_similarity_score("apple", "zebra")
        assert score < 0.2

class TestCronParsing:
    def test_valid_cron_expression(self):
        cron = "0 9 * * *"  # 9 AM daily
        parsed = parse_cron_expression(cron)
        assert parsed["hour"] == 9
        assert parsed["minute"] == 0

    def test_invalid_cron_raises_exception(self):
        with pytest.raises(ValueError):
            parse_cron_expression("invalid cron")

class TestDocumentFilename:
    def test_filename_generation_includes_timestamp(self):
        filename = generate_document_filename("report")
        assert filename.startswith("report_")
        assert filename.endswith(".pdf")
```

Listing 3.3: Service utility function tests

3.1.4 NLP Pipeline Unit Tests

```python
import pytest
from nlp.summarizer import Summarizer
from nlp.classifier import TopicClassifier
from nlp.embeddings import EmbeddingGenerator
from nlp.keywords import KeywordExtractor

class TestSummarizer:
    def test_summarizer_output_length(self):
        text = "Long article " * 100
        summarizer = Summarizer()
        summary = summarizer.summarize(text, max_length=100)
        assert len(summary) <= 100
        assert len(summary) > 0

```

```python
class TestTopicClassifier:
    def test_classifier_returns_valid_topic(self):
        classifier = TopicClassifier()
        text = "Breaking news about quantum computing advances"
        topic = classifier.classify(text)
        assert topic in ["AI", "Quantum", "Computing", "Science"]

    def test_classifier_confidence_score(self):
        classifier = TopicClassifier()
        result = classifier.classify("Test article", return_confidence=True)
        assert "topic" in result
        assert "confidence" in result
        assert 0 <= result["confidence"] <= 1

class TestKeywordExtractor:
    def test_extractor_minimum_keywords(self):
        extractor = KeywordExtractor()
        text = "This is a test article about AI and machine learning"
        keywords = extractor.extract(text, min_keywords=3)
        assert len(keywords) >= 3

class TestEmbeddingGenerator:
    def test_embedding_dimensions_for_ada_002(self):
        generator = EmbeddingGenerator(model="text-embedding-ada-002")
        text = "Sample text for embedding"
        embedding = generator.generate(text)
        assert len(embedding) == 1536  # Ada-002 produces 1536-dim embeddings

    def test_embedding_consistency(self):
        generator = EmbeddingGenerator()
        text = "Consistent text"
        emb1 = generator.generate(text)
        emb2 = generator.generate(text)
        assert emb1 == emb2  # Same input should produce same embedding
```

Listing 3.4: NLP pipeline tests

3.1.5 Celery Task Unit Tests

```python
import pytest
from unittest.mock import patch, MagicMock
from celery import current_app
from tasks.scrapers import scrape_hackernews_task
from tasks.nlp import process_article_nlp_task
from tasks.workflows import execute_workflow_task

@pytest.mark.celery
@patch('integrations.hackernews.fetch_stories')
def test_scrape_hackernews_task_mocked(mock_fetch):
    mock_fetch.return_value = [
        {
            "id": 1,
            "title": "AI Breakthrough",
            "url": "https://example.com/ai",
            "by": "user123"
        }
    ]

```

```
    result = scrape_hackernews_task.delay()
    assert result.get() == {"articles_created": 1}

@pytest.mark.celery
@patch('nlp.embeddings.EmbeddingGenerator')
def test_nlp_processing_task_updates_article(mock_embedding_gen):
    mock_embedding_gen.return_value.generate.return_value = [0.1] * 1536

    from articles.factories import ArticleFactory
    article = ArticleFactory(summary="Article to process")

    result = process_article_nlp_task.delay(article.id)
    article.refresh_from_db()

    assert article.embedding is not None
    assert article.keywords is not None
    assert article.topic is not None

@pytest.mark.celery
def test_workflow_execution_task_creates_run_record():
    from workflows.factories import WorkflowFactory
    from workflows.models import WorkflowRun

    workflow = WorkflowFactory()
    result = execute_workflow_task.delay(workflow.id)

    run = WorkflowRun.objects.get(workflow=workflow)
    assert run.status in ["pending", "running", "completed"]
```

Listing 3.5: Celery task unit tests with mocking

3.1.6 Agent Tool Unit Tests

```
import pytest
from unittest.mock import patch, MagicMock
from agents.tools import PDFGenerator, PowerPointGenerator, KnowledgeBaseSearch

class TestPDFGenerator:
    def test_generate_pdf_creates_file(self, tmp_path):
        generator = PDFGenerator()
        pdf_path = generator.generate(
            title="Test Report",
            content="Test content",
            output_dir=str(tmp_path)
        )
        assert pdf_path.exists()
        assert pdf_path.suffix == ".pdf"

class TestPowerPointGenerator:
    def test_generate_ppt_slide_count(self, tmp_path):
        generator = PowerPointGenerator()
        slides_data = [
            {"title": "Slide 1", "content": "Content 1"},
            {"title": "Slide 2", "content": "Content 2"},
            {"title": "Slide 3", "content": "Content 3"}
        ]
        ppt_path = generator.generate(
            slides=slides_data,
```

```python
            output_dir=str(tmp_path)
        )

        from pptx import Presentation
        prs = Presentation(ppt_path)
        assert len(prs.slides) == 3

class TestKnowledgeBaseSearch:
    @patch('agents.tools.VectorStore.similarity_search')
    def test_search_returns_results(self, mock_search):
        mock_search.return_value = [
            {"article_id": 1, "content": "Result 1", "score": 0.95},
            {"article_id": 2, "content": "Result 2", "score": 0.87}
        ]

        searcher = KnowledgeBaseSearch()
        results = searcher.search("test query", top_k=2)

        assert len(results) == 2
        assert results[0]["score"] > results[1]["score"]
```

Listing 3.6: AI agent tool tests

3.2 Frontend Unit Tests

3.2.1 React Component Tests

```python
import { render, screen, fireEvent, waitFor } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import ArticleCard from '@/components/ArticleCard';
import SearchBar from '@/components/SearchBar';
import ChatMessage from '@/components/ChatMessage';
import TrendChart from '@/components/TrendChart';
import WorkflowForm from '@/components/WorkflowForm';

describe('ArticleCard Component', () => {
  const mockArticle = {
    id: '1',
    title: 'AI Trends 2024',
    summary: 'Latest AI developments',
    source: 'TechNews',
    topic: 'AI',
    published_at: '2024-01-15'
  };

  test('renders article with all props', () => {
    render(<ArticleCard article={mockArticle} />);
    expect(screen.getByText('AI Trends 2024')).toBeInTheDocument();
    expect(screen.getByText('TechNews')).toBeInTheDocument();
  });

  test('bookmark button toggles state', async () => {
    const mockOnBookmark = jest.fn();
    render(
      <ArticleCard
        article={mockArticle}
        onBookmark={mockOnBookmark}
```

```
      />
    );

    const bookmarkBtn = screen.getByRole('button', { name: /bookmark/i });
    await userEvent.click(bookmarkBtn);
    expect(mockOnBookmark).toHaveBeenCalledWith('1');
  });
});

describe('SearchBar Component', () => {
  test('debounces search input', async () => {
    const mockOnSearch = jest.fn();
    render(<SearchBar onSearch={mockOnSearch} debounceMs={300} />);

    const input = screen.getByRole('textbox');
    await userEvent.type(input, 'AI');

    // Should not be called yet (still debouncing)
    expect(mockOnSearch).not.toHaveBeenCalled();

    // Wait for debounce
    await waitFor(() => {
      expect(mockOnSearch).toHaveBeenCalledWith('AI');
    }, { timeout: 500 });
  });
});

describe('ChatMessage Component', () => {
  test('renders user message with correct styling', () => {
    const userMsg = { role: 'user', content: 'Hello AI' };
    render(<ChatMessage message={userMsg} />);
    expect(screen.getByText('Hello AI')).toBeInTheDocument();
  });

  test('renders AI message with sources', () => {
    const aiMsg = {
      role: 'assistant',
      content: 'AI response',
      sources: [{ article_id: '1', relevance: 0.95 }]
    };
    render(<ChatMessage message={aiMsg} />);
    expect(screen.getByText('AI response')).toBeInTheDocument();
    expect(screen.getByText(/sources/i)).toBeInTheDocument();
  });
});

describe('TrendChart Component', () => {
  test('renders with data points', () => {
    const data = [
      { date: '2024-01-01', count: 10 },
      { date: '2024-01-02', count: 15 }
    ];
    render(<TrendChart data={data} />);
    expect(screen.getByRole('img')).toBeInTheDocument();
  });
});

describe('WorkflowForm Component', () => {
  test('validates required fields', async () => {
```

```
    const mockOnSubmit = jest.fn();
    render(<WorkflowForm onSubmit={mockOnSubmit} />);

    const submitBtn = screen.getByRole('button', { name: /submit/i });
    await userEvent.click(submitBtn);

    expect(screen.getByText(/name is required/i)).toBeInTheDocument();
    expect(mockOnSubmit).not.toHaveBeenCalled();
  });

  test('submits valid form data', async () => {
    const mockOnSubmit = jest.fn();
    render(<WorkflowForm onSubmit={mockOnSubmit} />);

    await userEvent.type(screen.getByLabelText(/name/i), 'New Workflow');
    await userEvent.type(screen.getByLabelText(/schedule/i), '0 9 * * *');
    await userEvent.click(screen.getByRole('button', { name: /submit/i }));

    expect(mockOnSubmit).toHaveBeenCalledWith({
      name: 'New Workflow',
      schedule: '0 9 * * *'
    });
  });
});
```

Listing 3.7: React component unit tests

3.2.2 Custom Hook Tests

```
import { renderHook, act, waitFor } from '@testing-library/react';
import { useAuth } from '@/hooks/useAuth';
import { useBookmarks } from '@/hooks/useBookmarks';
import { useSearch } from '@/hooks/useSearch';
import { useDarkMode } from '@/hooks/useDarkMode';

describe('useAuth Hook', () => {
  test('returns correct initial state', () => {
    const { result } = renderHook(() => useAuth());
    expect(result.current.user).toBeNull();
    expect(result.current.isLoading).toBe(true);
    expect(result.current.isAuthenticated).toBe(false);
  });

  test('login updates auth state', async () => {
    const { result } = renderHook(() => useAuth());

    await act(async () => {
      await result.current.login('user@test.com', 'password');
    });

    expect(result.current.user).not.toBeNull();
    expect(result.current.isAuthenticated).toBe(true);
  });
});

describe('useBookmarks Hook', () => {
  test('add and remove bookmarks', async () => {
    const { result } = renderHook(() => useBookmarks());
```

```

    await act(async () => {
      await result.current.addBookmark('1');
    });
    expect(result.current.isBookmarked('1')).toBe(true);

    await act(async () => {
      await result.current.removeBookmark('1');
    });
    expect(result.current.isBookmarked('1')).toBe(false);
  });
});

describe('useSearch Hook', () => {
  test('debounce behavior', async () => {
    const { result } = renderHook(() => useSearch({ debounceMs: 300 }));

    act(() => {
      result.current.search('query1');
    });

    expect(result.current.isSearching).toBe(false);

    await waitFor(() => {
      expect(result.current.results).toHaveLength(0);
    });
  });
});

describe('useDarkMode Hook', () => {
  test('toggles dark mode and persists', () => {
    const { result } = renderHook(() => useDarkMode());
    const initialMode = result.current.isDark;

    act(() => {
      result.current.toggle();
    });

    expect(result.current.isDark).toBe(!initialMode);
    expect(localStorage.getItem('darkMode')).toBe(String(!initialMode));
  });
});
```

Listing 3.8: Custom React hook tests

3.2.3 Utility Function Tests

```
import {
  formatRelativeTime,
  truncateText,
  parseTopics,
  buildApiUrl
} from '@/utils/helpers';

describe('formatRelativeTime', () => {
  test('formats recent dates correctly', () => {
    const now = new Date();
    const fiveMinutesAgo = new Date(now.getTime() - 5 * 60000);
```

```
    expect(formatRelativeTime(fiveMinutesAgo)).toBe('5 minutes ago');
  });

  test('formats older dates correctly', () => {
    const date = new Date('2024-01-01');
    const result = formatRelativeTime(date);
    expect(result).toMatch(/\d+ days ago/);
  });
});

describe('truncateText', () => {
  test('truncates text exceeding max length', () => {
    const text = 'This is a long text that needs truncation';
    const result = truncateText(text, 10);
    expect(result.length).toBeLessThanOrEqual(13); // 10 + '...'
    expect(result).toContain('...');
  });

  test('does not truncate short text', () => {
    const text = 'Short';
    const result = truncateText(text, 10);
    expect(result).toBe('Short');
  });
});

describe('parseTopics', () => {
  test('parses topic string correctly', () => {
    const topicString = 'AI, Machine Learning, Deep Learning';
    const topics = parseTopics(topicString);
    expect(topics).toEqual(['AI', 'Machine Learning', 'Deep Learning']);
  });

  test('handles single topic', () => {
    const topics = parseTopics('AI');
    expect(topics).toEqual(['AI']);
  });
});

describe('buildApiUrl', () => {
  test('builds URL with base and path', () => {
    const url = buildApiUrl('/articles', { page: 1, limit: 20 });
    expect(url).toContain('/api/v1/articles');
    expect(url).toContain('page=1');
    expect(url).toContain('limit=20');
  });
});
```

Listing 3.9: Frontend utility function tests

Chapter 4

Integration Testing

4.1 API Integration Tests

Integration tests verify that API endpoints work correctly end-to-end with the database, cache, and business logic.

4.1.1 Authentication Endpoints

```python
import pytest
from django.test import Client
from django.contrib.auth import get_user_model
from articles.factories import UserFactory

User = get_user_model()

@pytest.mark.django_db
class TestAuthenticationAPI:
    def setup_method(self):
        self.client = Client()
        self.register_url = '/api/v1/auth/register/'
        self.login_url = '/api/v1/auth/login/'

    def test_register_success_returns_201_with_token(self):
        response = self.client.post(self.register_url, {
            'username': 'newuser',
            'email': 'new@example.com',
            'password': 'SecurePass123!'
        }, content_type='application/json')

        assert response.status_code == 201
        data = response.json()
        assert 'access_token' in data
        assert 'refresh_token' in data

    def test_register_duplicate_email_returns_400(self):
        UserFactory(email='taken@example.com')
        response = self.client.post(self.register_url, {
            'username': 'newuser',
            'email': 'taken@example.com',
            'password': 'SecurePass123!'
        }, content_type='application/json')

```

```
        assert response.status_code == 400

    def test_login_success_returns_200_with_tokens(self):
        user = UserFactory(username='testuser')
        user.set_password('SecurePass123!')
        user.save()

        response = self.client.post(self.login_url, {
            'email': user.email,
            'password': 'SecurePass123!'
        }, content_type='application/json')

        assert response.status_code == 200
        data = response.json()
        assert 'access_token' in data
        assert 'refresh_token' in data

    def test_login_wrong_password_returns_401(self):
        user = UserFactory()
        user.set_password('CorrectPassword123!')
        user.save()

        response = self.client.post(self.login_url, {
            'email': user.email,
            'password': 'WrongPassword123!'
        }, content_type='application/json')

        assert response.status_code == 401

    def test_protected_endpoint_without_token_returns_401(self):
        response = self.client.get('/api/v1/articles/')
        assert response.status_code == 401

    def test_protected_endpoint_with_expired_token_returns_401(self):
        # Implementation would include mocked expired token
        pass
```

Listing 4.1: Authentication API integration tests

4.1.2 Articles Endpoints

```
@pytest.mark.django_db
class TestArticlesAPI:
    def setup_method(self):
        self.client = Client()
        self.user = UserFactory()
        self.articles_url = '/api/v1/articles/'

    def _authenticate(self):
        # Helper to add auth headers
        pass

    def test_list_articles_pagination(self):
        for i in range(25):
            ArticleFactory()

        response = self.client.get(f'{self.articles_url}?page=1&page_size=10')
        assert response.status_code == 200
```

```
        data = response.json()
        assert len(data['results']) == 10
        assert data['total'] == 25

    def test_list_articles_filter_by_topic(self):
        topic = TopicFactory(name='AI')
        ArticleFactory(topic=topic)
        ArticleFactory(topic=TopicFactory(name='Quantum'))

        response = self.client.get(f'{self.articles_url}?topic=AI')
        assert response.status_code == 200
        data = response.json()
        assert len(data['results']) == 1

    def test_get_article_detail_success(self):
        article = ArticleFactory()
        response = self.client.get(f'{self.articles_url}{article.id}/')

        assert response.status_code == 200
        data = response.json()
        assert data['title'] == article.title
        assert data['url'] == article.url

    def test_get_article_detail_not_found(self):
        response = self.client.get(f'{self.articles_url}99999/')
        assert response.status_code == 404
```

Listing 4.2: Articles API integration tests

4.2 Database Integration Tests

```
@pytest.mark.django_db
class TestDatabaseIntegration:
    def test_article_saved_to_db_with_correct_fields(self):
        article = ArticleFactory(
            title="Test Article",
            url="https://example.com/test"
        )
        article.refresh_from_db()

        assert Article.objects.count() == 1
        assert article.title == "Test Article"

    def test_article_embedding_stored_in_pgvector(self):
        embedding = [0.1] * 1536
        article = ArticleFactory(embedding=embedding)

        retrieved = Article.objects.get(id=article.id)
        assert retrieved.embedding is not None
        assert len(retrieved.embedding) == 1536

    def test_user_cascade_delete_removes_bookmarks(self):
        user = UserFactory()
        article = ArticleFactory()
        user.bookmarks.add(article)

        assert user.bookmarks.count() == 1
```

```
        user.delete()

        assert User.objects.filter(id=user.id).count() == 0

    def test_concurrent_article_insert_deduplication(self):
        url = "https://example.com/unique"
        ArticleFactory(url=url)

        # Attempting to insert duplicate should be rejected
        with pytest.raises(Exception):
            ArticleFactory(url=url)
```

Listing 4.3: Database integration tests

4.3 Cache Integration Tests

```
import pytest
from django.core.cache import cache
from articles.models import Article
from articles.factories import ArticleFactory

@pytest.mark.django_db
class TestCacheIntegration:
    def setup_method(self):
        cache.clear()

    def test_trending_articles_cached_in_redis(self):
        articles = [ArticleFactory() for _ in range(5)]

        # First call hits database
        cache.set('trending_articles', articles)

        # Second call hits cache
        cached = cache.get('trending_articles')
        assert cached is not None
        assert len(cached) == 5

    def test_cache_invalidated_on_new_article(self):
        cache.set('trending_articles', ['old_data'])

        # Create new article (should trigger cache invalidation)
        ArticleFactory()

        cached = cache.get('trending_articles')
        assert cached is None or cached != ['old_data']

    def test_rate_limit_enforced_via_redis(self):
        from django_ratelimit.decorators import ratelimit

        # Simulate 11 requests to endpoint limited to 10/minute
        for i in range(11):
            response = self.client.get('/api/v1/search/')
            if i < 10:
                assert response.status_code in [200, 400]
            else:
                assert response.status_code == 429  # Too Many Requests
```

Listing 4.4: Cache and Redis integration tests

4.4 Celery Integration Tests

```
@pytest.mark.django_db
class TestCeleryIntegration:
    def test_scraper_task_stores_articles_in_db(self):
        with patch('integrations.hackernews.fetch_stories') as mock_fetch:
            mock_fetch.return_value = [
                {'title': 'Article 1', 'url': 'http://example.com/1'},
                {'title': 'Article 2', 'url': 'http://example.com/2'}
            ]

            from tasks.scrapers import scrape_hackernews_task
            scrape_hackernews_task()

            assert Article.objects.count() == 2

    def test_nlp_task_updates_article_fields(self):
        article = ArticleFactory(embedding=None, keywords=None)

        from tasks.nlp import process_article_nlp_task
        process_article_nlp_task(article.id)

        article.refresh_from_db()
        assert article.embedding is not None
        assert article.keywords is not None

    def test_workflow_execution_creates_run_record(self):
        from workflows.factories import WorkflowFactory
        from workflows.models import WorkflowRun

        workflow = WorkflowFactory()
        from tasks.workflows import execute_workflow_task
        execute_workflow_task(workflow.id)

        run = WorkflowRun.objects.filter(workflow=workflow).first()
        assert run is not None
        assert run.status in ['pending', 'completed', 'failed']
```

Listing 4.5: Celery task integration tests

Chapter 5

End-to-End Testing with Playwright

E2E tests verify complete user workflows from login to final action. These tests use Playwright for cross-browser automation.

5.1 Authentication Flows

```
import { test, expect } from '@playwright/test';

test.describe('Authentication E2E Tests', () => {
  test('register, login, and logout flow', async ({ page }) => {
    // Register new user
    await page.goto('http://localhost:3000/register');
    await page.fill('input[name="email"]', 'e2euser@test.com');
    await page.fill('input[name="password"]', 'SecurePass123!');
    await page.fill('input[name="confirmPassword"]', 'SecurePass123!');
    await page.click('button[type="submit"]');

    // Should redirect to dashboard
    await expect(page).toHaveURL(/.*\/dashboard/);
    await expect(page.locator('h1')).toContainText('Dashboard');

    // Logout
    await page.click('[data-testid="user-menu"]');
    await page.click('button:has-text("Logout")');

    // Should redirect to login
    await expect(page).toHaveURL(/.*\/login/);
  });

  test('password reset flow', async ({ page }) => {
    await page.goto('http://localhost:3000/login');
    await page.click('a:has-text("Forgot password")');

    await page.fill('input[name="email"]', 'user@example.com');
    await page.click('button:has-text("Send reset link")');

    await expect(page.locator('text=Check your email')).toBeVisible();
  });

  test('protected route redirects to login', async ({ page }) => {
    await page.goto('http://localhost:3000/dashboard');
    await expect(page).toHaveURL(/.*\/login/);
```

```
  });
});
```

Listing 5.1: Playwright E2E authentication tests

5.2 Dashboard and Article Flows

```
test.describe('Dashboard E2E Tests', () => {
  test('dashboard loads and displays articles', async ({ page, context }) => {
    // Login first
    const cookies = await context.cookies();
    await page.context().addCookies(cookies);

    await page.goto('http://localhost:3000/dashboard');

    // Wait for articles to load
    const articles = page.locator('[data-testid="article-card"]');
    await expect(articles.first()).toBeVisible();
  });

  test('click article opens detail page', async ({ page }) => {
    await page.goto('http://localhost:3000/dashboard');

    const firstArticle = page.locator('[data-testid="article-card"]').first();
    const articleTitle = await firstArticle.locator('h2').textContent();

    await firstArticle.click();

    await expect(page.locator('h1')).toContainText(articleTitle);
    await expect(page.locator('text=Source:')).toBeVisible();
  });

  test('bookmark article appears in library', async ({ page }) => {
    await page.goto('http://localhost:3000/dashboard');

    const bookmarkBtn = page.locator('[data-testid="article-card"]')
      .first()
      .locator('[data-testid="bookmark-btn"]');
    await bookmarkBtn.click();

    await page.goto('http://localhost:3000/library');
    const bookmarkedArticles = page.locator('[data-testid="article-card"]');
    await expect(bookmarkedArticles.first()).toBeVisible();
  });

  test('search functionality returns results', async ({ page }) => {
    await page.goto('http://localhost:3000/dashboard');

    const searchInput = page.locator('input[placeholder="Search articles..."]');
    await searchInput.fill('artificial intelligence');
    await page.waitForLoadState('networkidle');

    const results = page.locator('[data-testid="article-card"]');
    const count = await results.count();
    expect(count).toBeGreaterThan(0);
  });
});
```

Listing 5.2: Dashboard E2E tests

5.3 AI Chat Flow

```
test.describe('AI Chat E2E Tests', () => {
  test('send message and receive response', async ({ page }) => {
    await page.goto('http://localhost:3000/chat');

    const input = page.locator('input[placeholder="Ask a question..."]');
    await input.fill('What are the latest AI developments?');
    await page.click('button[type="submit"]');

    // Wait for response
    const response = page.locator('[data-testid="chat-message"][role="assistant"]');
    await expect(response).toBeVisible({ timeout: 10000 });
  });

  test('chat sources displayed with articles', async ({ page }) => {
    await page.goto('http://localhost:3000/chat');

    const input = page.locator('input[placeholder="Ask a question..."]');
    await input.fill('artificial intelligence');
    await page.click('button[type="submit"]');

    // Sources should appear
    const sources = page.locator('[data-testid="chat-source"]');
    await expect(sources.first()).toBeVisible({ timeout: 10000 });
  });

  test('new conversation started fresh', async ({ page }) => {
    await page.goto('http://localhost:3000/chat');

    const input = page.locator('input[placeholder="Ask a question..."]');
    await input.fill('First message');
    await page.click('button[type="submit"]');

    await page.click('button:has-text("New Conversation")');

    const messages = page.locator('[data-testid="chat-message"]');
    expect(await messages.count()).toBe(0);
  });
});
```

Listing 5.3: AI Chat E2E tests

5.4 Workflow Automation E2E

```
test.describe('Workflow Automation E2E Tests', () => {
  test('create workflow form submission', async ({ page }) => {
    await page.goto('http://localhost:3000/workflows');
    await page.click('button:has-text("Create Workflow")');

    await page.fill('input[name="name"]', 'Daily AI News');
```

```
    await page.fill('input[name="schedule"]', '0 9 * * *');
    await page.click('button[type="submit"]');

    await expect(page.locator('text=Workflow created successfully')).toBeVisible();
  });

  test('trigger workflow manually', async ({ page }) => {
    await page.goto('http://localhost:3000/workflows');

    const workflowRow = page.locator('text=Daily AI News').first();
    await page.locator('[data-testid="trigger-btn"]', { has: workflowRow }).click();

    await expect(page.locator('text=Workflow triggered')).toBeVisible();
  });

  test('workflow run history displayed', async ({ page }) => {
    await page.goto('http://localhost:3000/workflows/1/history');

    const runs = page.locator('[data-testid="workflow-run"]');
    await expect(runs.first()).toBeVisible();
  });
});
```

Listing 5.4: Workflow automation E2E tests

Chapter 6

Performance Testing with Locust

Performance testing ensures the platform meets response time and throughput targets under various load conditions.

6.1 Load Test Scenarios

6.1.1 Scenario 1: Normal Load

Normal load simulates 50 concurrent users over 10 minutes with realistic traffic distribution.

```python
from locust import HttpUser, task, between, events
import random

class NormalLoadUser(HttpUser):
    wait_time = between(1, 3)

    def on_start(self):
        self.token = self.login()

    def login(self):
        response = self.client.post(
            "/api/v1/auth/login/",
            json={"email": "user@example.com", "password": "password"},
            name="login"
        )
        return response.json().get('access_token')

    @task(60)
    def browse_dashboard(self):
        # 60% of traffic
        page = random.randint(1, 5)
        self.client.get(
            f"/api/v1/articles/?page={page}&page_size=20",
            headers={"Authorization": f"Bearer {self.token}"},
            name="GET /api/v1/articles/"
        )

    @task(20)
    def semantic_search(self):
        # 20% of traffic
        queries = ["AI", "ML", "quantum", "blockchain", "robotics"]
        self.client.post(
            "/api/v1/search/semantic/",
```

```
            json={"query": random.choice(queries)},
            headers={"Authorization": f"Bearer {self.token}"},
            name="POST /api/v1/search/semantic/"
        )

    @task(10)
    def chat(self):
        # 10% of traffic
        self.client.post(
            "/api/v1/ai/chat/",
            json={"message": "What are latest trends?"},
            headers={"Authorization": f"Bearer {self.token}"},
            name="POST /api/v1/ai/chat/"
        )

    @task(10)
    def bookmark_article(self):
        # 10% of traffic
        article_id = random.randint(1, 10000)
        self.client.post(
            f"/api/v1/articles/{article_id}/bookmark/",
            headers={"Authorization": f"Bearer {self.token}"},
            name="POST /api/v1/articles/{id}/bookmark/"
        )
```

Listing 6.1: Locust normal load scenario

6.1.2 Scenario 2: Peak Load and Stress Test

```
class PeakLoadUser(HttpUser):
    wait_time = between(0.5, 2)
    # Same tasks as normal load but higher request rate

class StressTestUser(HttpUser):
    wait_time = between(0.1, 1)
    # Even higher concurrency and request rate

# Locust configuration for different scenarios
import logging
from locust import LoadTestShape

class StagesShape(LoadTestShape):
    stages = [
        {"duration": 120, "users": 50, "spawn_rate": 5},      # Ramp up
        {"duration": 300, "users": 50, "spawn_rate": 0},      # Hold
        {"duration": 120, "users": 200, "spawn_rate": 10},    # Peak load
        {"duration": 300, "users": 200, "spawn_rate": 0},     # Hold peak
        {"duration": 60, "users": 500, "spawn_rate": 50},     # Spike test
        {"duration": 120, "users": 500, "spawn_rate": 0},     # Hold spike
    ]
```

Listing 6.2: Peak load and stress test configuration

6.2 Performance Targets

- **p50 response time**: ¡ 200ms

- **p95 response time**: ¡ 500ms
- **p99 response time**: ¡ 2000ms
- **Error rate at normal load**: ¡ 1%
- **Error rate at peak load**: ¡ 5%
- **Throughput**: ¿ 100 requests/second

6.3 Frontend Performance Testing

Frontend performance is measured using Lighthouse CI and Core Web Vitals:

```
{
  "ci": {
    "assert": {
      "preset": "lighthouse:recommended",
      "assertions": {
        "first-contentful-paint": ["error", {"maxNumericValue": 2500}],
        "largest-contentful-paint": ["error", {"maxNumericValue": 2500}],
        "cumulative-layout-shift": ["error", {"maxNumericValue": 0.1}],
        "speed-index": ["error", {"maxNumericValue": 3500}]
      }
    },
    "collect": {
      "url": ["http://localhost:3000/dashboard"],
      "numberOfRuns": 3
    }
  }
}
```

Listing 6.3: Lighthouse CI configuration

6.4 Database Query Optimization

Backend performance relies on efficient database queries. We use `django-debug-toolbar` for development and `EXPLAIN ANALYZE` for production query analysis:

```
# Problem: N+1 query - loads user for each article
articles = Article.objects.all()
for article in articles:
    print(article.user.name)  # SLOW: One query per article

# Solution: Use select_related for foreign keys
articles = Article.objects.select_related('user').all()
for article in articles:
    print(article.user.name)  # FAST: Single query

# For many-to-many, use prefetch_related
articles = Article.objects.prefetch_related('bookmarks').all()
for article in articles:
    print(article.bookmarks.count())  # FAST: Two queries total
```

Listing 6.4: Query optimization example

Chapter 7

Security Testing

Security testing ensures protection against common vulnerabilities and attacks. We employ a multi-layered approach combining static analysis, dynamic scanning, and manual testing.

7.1 Static Analysis with bandit

Python static security analysis identifies common security issues:

```
# Install bandit
pip install bandit

# Run security scan on all Python code
bandit -r apps/ -f json -o bandit-report.json

# Example: bandit detects hardcoded secrets, SQL injection risks
# [MEDIUM] hardcoded_sql_string: Possible SQL injection
# [HIGH] hardcoded_password_string: Potential hardcoded password
```

Listing 7.1: bandit static security scanning

7.2 Dynamic Security Scanning with OWASP ZAP

OWASP ZAP performs active vulnerability scanning on running application:

```
docker run -t owasp/zap2docker-stable zap-baseline.py \
  -t http://localhost:8000 \
  -r zap-report.html \
  -J zap-report.json

# Checks for: XSS, SQL Injection, CSRF, Insecure Deserialization
```

Listing 7.2: OWASP ZAP baseline scan in CI

7.3 Dependency Vulnerability Scanning

```
pip install safety
safety check --json > safety-report.json

# Also check JavaScript dependencies
npm audit --json > npm-audit.json
```

```
yarn audit --json > yarn-audit.json
```

Listing 7.3: Python dependency security check

7.4 Manual Security Tests

```python
import pytest
from django.test import Client

@pytest.mark.django_db
class TestSecurityManual:
    def test_sql_injection_protection(self):
        # Attempt SQL injection in search endpoint
        client = Client()
        response = client.get("/api/v1/articles/?search='; DROP TABLE articles; --")
        assert response.status_code in [200, 400, 422]
        # Articles table should still exist
        from articles.models import Article
        Article.objects.count()  # Should not raise

    def test_xss_input_sanitization(self):
        # Attempt XSS payload in article creation
        client = Client()
        response = client.post("/api/v1/articles/", {
            "title": "<script>alert('XSS')</script>",
            "url": "http://example.com",
            "summary": "<img src=x onerror=alert('XSS')>"
        })
        # Should sanitize or reject
        assert response.status_code in [400, 422]

    def test_authentication_bypass(self):
        # Attempt to access protected resource without token
        client = Client()
        response = client.get("/api/v1/articles/")
        assert response.status_code == 401

    def test_idor_access_control(self):
        # Attempt to access another user's resources
        from articles.factories import UserFactory
        user1 = UserFactory()
        user2 = UserFactory()
        article = Article.objects.create(owner=user1, ...)

        # User2 attempts to delete user1's article
        # Should be rejected with 403 Forbidden
        pass

    def test_rate_limiting_enforced(self):
        # Attempt to exceed rate limit
        client = Client()
        for i in range(101):  # 100+ requests
            response = client.post("/api/v1/search/")
            if i == 100:
                assert response.status_code == 429  # Too Many Requests
```

Listing 7.4: Manual security test cases

Chapter 8

AI and ML Testing

AI/ML components require specialized testing to ensure model quality, consistency, and safety.

8.1 Summarization Quality Tests

```python
import pytest
from nlp.summarizer import Summarizer
from rouge_score import rouge_scorer

class TestSummarizationQuality:
    def test_summarizer_rouge_score(self):
        summarizer = Summarizer()

        text = """
        Machine learning is a subset of artificial intelligence.
        It enables computers to learn from data without explicit programming.
        Deep learning uses neural networks with multiple layers.
        """

        reference_summary = "Machine learning uses neural networks to learn from data."
        generated_summary = summarizer.summarize(text, max_length=50)

        scorer = rouge_scorer.RougeScorer(['rougeL'], use_stemmer=True)
        scores = scorer.score(reference_summary, generated_summary)

        # ROUGE-L score should be >= 0.4
        assert scores['rougeL'].fmeasure >= 0.4

    def test_summarizer_preserves_key_facts(self):
        summarizer = Summarizer()
        text = "The CEO announced 1000 new jobs in Q1 2024."
        summary = summarizer.summarize(text)

        # Should preserve critical numbers
        assert "1000" in summary or "jobs" in summary
```

Listing 8.1: NLP summarization quality tests

8.2 Topic Classification Tests

```python
class TestTopicClassification:
    def test_classifier_accuracy_on_test_set(self):
        classifier = TopicClassifier()

        test_cases = [
            ("Quantum computers achieve new milestone", "Quantum"),
            ("Neural networks improve vision tasks", "AI"),
            ("Bitcoin price surges", "Crypto"),
        ]

        correct = 0
        for text, expected_topic in test_cases:
            predicted = classifier.classify(text)
            if predicted == expected_topic:
                correct += 1

        accuracy = correct / len(test_cases)
        # Should achieve >= 85% accuracy
        assert accuracy >= 0.85
```

Listing 8.2: Topic classification accuracy tests

8.3 Semantic Search Relevance Tests

```python
class TestSemanticSearchRelevance:
    @pytest.mark.django_db
    def test_mrr_score_on_queries(self):
        from articles.factories import ArticleFactory
        from search.semantic import SemanticSearch

        # Create test articles
        articles = [
            ArticleFactory(title="GPT-4 Released", summary="OpenAI releases GPT-4"),
            ArticleFactory(title="Climate Change", summary="Global warming accelerates"),
            ArticleFactory(title="AI Ethics", summary="Responsible AI development"),
        ]

        search = SemanticSearch()
        query = "artificial intelligence"
        results = search.search(query, top_k=3)

        # Compute Mean Reciprocal Rank
        mrr = 0
        for i, result in enumerate(results, 1):
            if 'AI' in result['title'] or 'AI' in result['summary']:
                mrr = 1 / i
                break

        # MRR should be >= 0.7
        assert mrr >= 0.7
```

Listing 8.3: Semantic search quality tests

8.4 Embedding Consistency Tests

```python
class TestEmbeddingConsistency:
    def test_same_text_produces_same_embedding(self):
        from nlp.embeddings import EmbeddingGenerator

        generator = EmbeddingGenerator(model="text-embedding-ada-002")
        text = "Machine learning advances"

        emb1 = generator.generate(text)
        emb2 = generator.generate(text)

        assert emb1 == emb2

    def test_similar_texts_have_high_cosine_similarity(self):
        from nlp.embeddings import EmbeddingGenerator
        import numpy as np

        generator = EmbeddingGenerator()

        text1 = "Machine learning is powerful"
        text2 = "ML is very powerful"
        text3 = "I like pizza"

        emb1 = np.array(generator.generate(text1))
        emb2 = np.array(generator.generate(text2))
        emb3 = np.array(generator.generate(text3))

        # Cosine similarity between similar texts
        sim_12 = np.dot(emb1, emb2) / (np.linalg.norm(emb1) * np.linalg.norm(emb2))
        sim_13 = np.dot(emb1, emb3) / (np.linalg.norm(emb1) * np.linalg.norm(emb3))

        assert sim_12 > sim_13
```

Listing 8.4: Embedding consistency and drift tests

8.5 RAG Hallucination Detection

```python
class TestRAGHallucination:
    @patch('langchain.llms.OpenAI')
    def test_response_grounded_in_sources(self, mock_llm):
        from agents.rag import RAGAgent

        sources = [
            {"content": "Tesla stock price is 250 dollars"},
            {"content": "Apple stock price is 150 dollars"}
        ]

        # Mock LLM response
        mock_llm.return_value = "Tesla stock is 250 dollars"

        agent = RAGAgent(sources=sources)
        response = agent.answer("What is Tesla stock price?")

        # Response should cite price from sources
        assert "250" in response or "Tesla" in response
        # Should not hallucinate new facts
        assert "Microsoft" not in response
```

Listing 8.5: RAG hallucination testing

Chapter 9

Test Data Management

9.1 Factory Definitions

Test factories provide consistent, realistic test data:

```
import factory
from faker import Faker
from django.contrib.auth import get_user_model

fake = Faker()
User = get_user_model()

class UserFactory(factory.django.DjangoModelFactory):
    class Meta:
        model = User

    username = factory.Sequence(lambda n: f"user_{n}")
    email = factory.LazyAttribute(lambda o: fake.email())
    first_name = factory.Faker('first_name')
    last_name = factory.Faker('last_name')
    is_active = True
    is_staff = False

    @factory.post_generation
    def password(obj, create, extracted, **kwargs):
        if not create:
            return
        obj.set_password(extracted or "DefaultPassword123!")

class TopicFactory(factory.django.DjangoModelFactory):
    class Meta:
        model = Topic

    name = factory.Sequence(lambda n: f"topic_{n}")
    description = factory.Faker('sentence')
    color = factory.Faker('hex_color')

class ArticleFactory(factory.django.DjangoModelFactory):
    class Meta:
        model = Article

    title = factory.Faker('sentence', nb_words=8)
    url = factory.LazyAttribute(lambda o: fake.url())
    summary = factory.Faker('paragraph', nb_sentences=5)
```

```
    content = factory.Faker('text', max_nb_chars=2000)
    source = factory.LazyAttribute(lambda o: fake.domain_name())
    topic = factory.SubFactory(TopicFactory)
    published_at = factory.Faker('date_time_this_year')
    created_at = factory.Faker('date_time_this_month')

class WorkflowFactory(factory.django.DjangoModelFactory):
    class Meta:
        model = Workflow

    name = factory.Sequence(lambda n: f"workflow_{n}")
    description = factory.Faker('sentence')
    cron_expression = "0 9 * * *"  # 9 AM daily
    is_active = True
    created_by = factory.SubFactory(UserFactory)
```

Listing 9.1: Comprehensive factory definitions

9.2 Test Database Isolation

```
# conftest.py
import pytest
from django.test.utils import override_settings

@pytest.fixture(scope="function")
def db_with_rollback(db):
    """Provides database with rollback for each test."""
    return db

@pytest.fixture
def authenticated_client(client, db):
    """Returns client authenticated as test user."""
    user = UserFactory(email="test@example.com")
    user.set_password("testpassword")
    user.save()
    client.login(username="test@example.com", password="testpassword")
    return client

@pytest.fixture
def mock_openai(monkeypatch):
    """Mocks OpenAI API calls."""
    def mock_embed(text):
        return [0.1] * 1536

    monkeypatch.setattr("openai.Embedding.create", mock_embed)
    return mock_embed
```

Listing 9.2: pytest-django database isolation configuration

9.3 Mock Strategies for External APIs

```
import responses
from unittest.mock import patch, MagicMock

@responses.activate
```

```python
def test_github_api_integration():
    # Mock GitHub API
    responses.add(
        responses.GET,
        'https://api.github.com/repos/owner/repo',
        json={'stars': 1000, 'forks': 500},
        status=200
    )

    from integrations.github import get_repo_stats
    stats = get_repo_stats('owner/repo')
    assert stats['stars'] == 1000

@patch('integrations.arxiv.fetch_papers')
def test_arxiv_scraper(mock_fetch):
    mock_fetch.return_value = [
        {'title': 'Paper 1', 'url': 'http://arxiv.org/1'},
        {'title': 'Paper 2', 'url': 'http://arxiv.org/2'}
    ]

    from integrations.arxiv import ArxivScraper
    scraper = ArxivScraper()
    papers = scraper.fetch()
    assert len(papers) == 2

@patch('integrations.youtube.search_videos')
def test_youtube_integration(mock_search):
    mock_search.return_value = [
        {'title': 'AI Tutorial', 'url': 'https://youtube.com/1'},
        {'title': 'ML Course', 'url': 'https://youtube.com/2'}
    ]

    results = mock_search('AI tutorials')
    assert len(results) == 2
```

Listing 9.3: External API mocking strategies

Chapter 10

CI/CD Test Integration

10.1 GitHub Actions Test Pipeline

```yaml
name: Test Suite

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]

jobs:
  backend-tests:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: synapse_test
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

      redis:
        image: redis:7
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
    - uses: actions/checkout@v3

    - name: Set up Python
      uses: actions/setup-python@v4
      with:
        python-version: '3.11'

```

```yaml
    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install -r requirements-test.txt

    - name: Lint with flake8
      run: |
        flake8 apps/ --count --select=E9,F63,F7,F82 --show-source --statistics

    - name: Run unit tests
      run: |
        pytest tests/unit/ -v --cov=apps --cov-report=term-missing

    - name: Run integration tests
      run: |
        pytest tests/integration/ -v --tb=short

    - name: Security scan with bandit
      run: |
        bandit -r apps/ -f json -o bandit-report.json

    - name: Upload coverage to Codecov
      uses: codecov/codecov-action@v3
      with:
        files: ./coverage.xml
        flags: unittests
        fail_ci_if_error: true

  frontend-tests:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v3

    - name: Set up Node
      uses: actions/setup-node@v3
      with:
        node-version: '18'

    - name: Install dependencies
      run: npm ci

    - name: Lint with ESLint
      run: npm run lint

    - name: Run Jest tests
      run: npm test -- --coverage

    - name: Upload coverage
      uses: codecov/codecov-action@v3
      with:
        files: ./coverage/coverage-final.json
        flags: frontend

  e2e-tests:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v3
```

```yaml

    - name: Set up Node
      uses: actions/setup-node@v3
      with:
        node-version: '18'

    - name: Install dependencies
      run: npm ci

    - name: Install Playwright
      run: npx playwright install --with-deps

    - name: Build frontend
      run: npm run build

    - name: Run E2E tests
      run: npm run test:e2e

    - name: Upload test results
      uses: actions/upload-artifact@v3
      if: always()
      with:
        name: playwright-report
        path: playwright-report/
```

Listing 10.1: GitHub Actions CI pipeline configuration

10.2 Test Parallelization

```bash
# Install pytest-xdist
pip install pytest-xdist

# Run tests in parallel using 4 workers
pytest -n 4

# Distribute by scope (each test file to separate worker)
pytest -n 4 --dist loadscope

# Generate JUnit XML for CI systems
pytest --junit-xml=junit.xml
```

Listing 10.2: Running tests in parallel with pytest-xdist

10.3 Test Failure Notifications

```yaml
- name: Notify Slack on failure
  if: failure()
  uses: slackapi/slack-github-action@v1.24.0
  with:
    webhook-url: ${{ secrets.SLACK_WEBHOOK }}
    payload: |
      {
        "text": "Test suite failed",
        "blocks": [
          {
```

```
            "type": "section",
            "text": {
              "type": "mrkdwn",
              "text": "[FAIL] Tests failed on branch ${{ github.ref }}\n${{ github.server_url }}/${{ github.repository }}/actions/runs/${{ github.run_id }}"
            }
          }
        ]
      }
```

Listing 10.3: Slack notification for test failures

Chapter 11

Test Environment Configuration

11.1 pytest Configuration

```
[tool.pytest.ini_options]
DJANGO_SETTINGS_MODULE = "config.settings.test"
python_files = ["test_*.py", "*_test.py"]
python_classes = ["Test*"]
python_functions = ["test_*"]
addopts = """
    --cov=apps
    --cov-report=html
    --cov-report=term-missing
    --cov-report=xml
    --cov-fail-under=80
    --strict-markers
    -ra
    --tb=short
"""
testpaths = ["tests"]
markers = [
    "django_db: marks tests as requiring database access",
    "celery: marks tests as celery task tests",
    "slow: marks tests as slow",
    "integration: marks tests as integration tests",
    "e2e: marks tests as end-to-end tests",
]
filterwarnings = [
    "ignore::DeprecationWarning",
]
```

Listing 11.1: pytest configuration in pyproject.toml

11.2 conftest.py Structure

```
import pytest
import django
from django.conf import settings
from django.test.utils import get_runner

@pytest.fixture(scope="session")
def django_db_setup():
```

```python
    """Override Django's default setup."""
    settings.DATABASES['default'] = {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'synapse_test',
        'USER': 'postgres',
        'PASSWORD': 'postgres',
        'HOST': 'localhost',
        'PORT': '5432',
    }

@pytest.fixture
def auth_client(client, django_db_setup):
    """Returns authenticated test client."""
    from articles.factories import UserFactory
    user = UserFactory(email="test@example.com")
    user.set_password("testpass123")
    user.save()
    client.login(username="test@example.com", password="testpass123")
    return client

@pytest.fixture
def mock_openai_embedding(monkeypatch):
    """Mocks OpenAI embedding API."""
    def mock_create(*args, **kwargs):
        return {"data": [{"embedding": [0.1] * 1536}]}
    monkeypatch.setattr("openai.Embedding.create", mock_create)

@pytest.fixture
def mock_celery_task(monkeypatch):
    """Mocks Celery task execution to run synchronously."""
    monkeypatch.setattr(
        "celery.app.task.Task.delay",
        lambda self, *args, **kwargs: self(*args, **kwargs)
    )
```

Listing 11.2: Comprehensive conftest.py with reusable fixtures

11.3 Jest Configuration

```javascript
module.exports = {
  preset: 'ts-jest',
  testEnvironment: 'jsdom',
  roots: ['<rootDir>/src'],
  testMatch: ['**/__tests__/**/*.ts?(x)', '**/?(*.)+(spec|test).ts?(x)'],
  moduleNameMapper: {
    '^@/(.*)$': '<rootDir>/src/$1',
  },
  setupFilesAfterEnv: ['<rootDir>/src/setupTests.ts'],
  collectCoverageFrom: [
    'src/**/*.{ts,tsx}',
    '!src/**/*.d.ts',
    '!src/index.tsx',
  ],
  coverageThreshold: {
    global: {
      branches: 70,
      functions: 70,
```

```
      lines: 70,
      statements: 70,
    },
  },
};

\section{Playwright Configuration}

\begin{lstlisting}[language=Java, caption=playwright.config.ts for E2E testing]
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests/e2e',
  testMatch: '**/*e2e.spec.ts',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 1 : undefined,
  reporter: 'html',
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
  },
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
  ],
  webServer: {
    command: 'npm run dev',
    url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
  },
});
```

Listing 11.3: jest.config.js for frontend testing

11.4 Docker Compose Test Override

```
version: '3.8'

services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: synapse_test
      POSTGRES_USER: synapse
      POSTGRES_PASSWORD: test_password
```

```yaml
    ports:
      - "5433:5432"

  redis:
    image: redis:7
    ports:
      - "6380:6379"

  backend:
    build: .
    command: pytest tests/
    environment:
      DATABASE_URL: postgresql://synapse:test_password@postgres:5432/synapse_test
      REDIS_URL: redis://redis:6379/1
      DEBUG: "False"
    depends_on:
      - postgres
      - redis

  frontend:
    build: ./frontend
    command: npm test
    ports:
      - "3001:3000"
```

Listing 11.4: docker-compose.test.yml for isolated testing

Chapter 12

Quality Metrics and Reporting

12.1 Coverage Targets by Module

Module	Target Coverage	Priority
Authentication	95%	Critical
Authorization	95%	Critical
Article Models	90%	Critical
Search (Semantic)	85%	High
Workflows	85%	High
NLP Pipeline	80%	High
Cache Layer	80%	Medium
Utilities	75%	Medium
Frontend Components	70%	Medium
Helpers/Utils	60%	Low

Table 12.1: Code coverage targets by module

12.2 Test Execution Time Budgets

- **Unit Tests**: Must complete in less than 2 minutes. Measured from `pytest tests/unit/`
- **Integration Tests**: Must complete in less than 10 minutes. Measured from `pytest tests/integration/`
- **End-to-End Tests**: Must complete in less than 30 minutes. Measured from `npm run test:e2e`
- **Full Test Suite**: Must complete in less than 45 minutes with parallelization
- **Performance Tests**: Must complete in less than 20 minutes per scenario

12.3 Quality Gates

Pull requests are automatically blocked if any of these quality gates fail:

1. Code coverage drops below target thresholds
2. Any high-severity security vulnerability detected
3. Linting fails (flake8, ESLint)
4. Critical tests fail

5. Commit messages do not follow convention
6. Type checking fails (mypy, TypeScript)

12.4 Bug Escape Rate Tracking

Bug escape rate measures defects that reach production. We track:

- Bugs found in development vs production (target: 95% found pre-release)
- Time to detect critical bugs (target: ¡ 1 hour via monitoring)
- Escape rate by test phase (unit, integration, E2E, production)
- Root cause analysis (missing test, flaky test, environment issue)

12.5 Test Health Dashboard

Key metrics tracked on test health dashboard:

1. **Pass Rate**: Percentage of tests that pass on first run (target: 99%)
2. **Flaky Tests**: Tests that pass/fail intermittently (action: quarantine and fix)
3. **Execution Time Trends**: Track if tests are getting slower
4. **Coverage Trends**: Track coverage changes over time
5. **Critical Path Duration**: Track time to get feedback after code push
6. **Test Maintenance**: Track test code changes vs product code changes (ratio should be ¡ 1:2)

Chapter 13

Accessibility and Compliance Testing

13.1 Automated Accessibility Testing

Jest-axe provides automated accessibility rule checking:

```
import { render } from '@testing-library/react';
import { axe, toHaveNoViolations } from 'jest-axe';
import ArticleCard from '@/components/ArticleCard';

expect.extend(toHaveNoViolations);

describe('ArticleCard Accessibility', () => {
  test('should not have any accessibility violations', async () => {
    const { container } = render(
      <ArticleCard article={mockArticle} />
    );
    const results = await axe(container);
    expect(results).toHaveNoViolations();
  });

  test('has proper heading hierarchy', () => {
    const { container } = render(<ArticleCard article={mockArticle} />);
    const headings = container.querySelectorAll('h1, h2, h3, h4, h5, h6');

    // Verify no skipped heading levels
    let previousLevel = 0;
    for (const heading of headings) {
      const level = parseInt(heading.tagName[1]);
      expect(level - previousLevel).toBeLessThanOrEqual(1);
      previousLevel = level;
    }
  });

  test('form labels properly associated', () => {
    const { getByLabelText } = render(<SearchForm />);
    const input = getByLabelText('Search articles');
    expect(input).toBeInTheDocument();
  });
});
```

Listing 13.1: Jest-axe accessibility tests

13.2 Keyboard Navigation E2E Tests

```
test.describe('Keyboard Navigation', () => {
  test('tab through main navigation', async ({ page }) => {
    await page.goto('http://localhost:3000/dashboard');

    // Start from top
    await page.keyboard.press('Tab');
    let focused = await page.evaluate(() => document.activeElement.id);
    expect(focused).toBe('main-nav-home');

    // Tab through menu items
    await page.keyboard.press('Tab');
    focused = await page.evaluate(() => document.activeElement.id);
    expect(focused).toBe('main-nav-articles');

    // Tab through search
    await page.keyboard.press('Tab');
    focused = await page.evaluate(() => document.activeElement.tagName);
    expect(focused).toBe('INPUT');
  });

  test('navigate article list with arrow keys', async ({ page }) => {
    await page.goto('http://localhost:3000/articles');

    const firstArticle = page.locator('[data-testid="article-card"]').first();
    await firstArticle.focus();

    // Press down arrow to select next
    await page.keyboard.press('ArrowDown');
    const secondArticle = page.locator('[data-testid="article-card"]').nth(1);

    const isFocused = await secondArticle.evaluate(
      (el) => el === document.activeElement
    );
    expect(isFocused).toBe(true);
  });

  test('open dropdown with Enter key', async ({ page }) => {
    await page.goto('http://localhost:3000/dashboard');

    const userMenu = page.locator('[data-testid="user-menu"]');
    await userMenu.focus();
    await page.keyboard.press('Enter');

    const dropdown = page.locator('[data-testid="user-menu-dropdown"]');
    await expect(dropdown).toBeVisible();
  });
});
```

Listing 13.2: Keyboard navigation E2E tests

13.3 Screen Reader Testing

```
describe('Screen Reader Support', () => {
  test('article cards have semantic HTML', () => {
    const { container } = render(<ArticleCard article={mockArticle} />);
```

```

    // Should use article element for semantic meaning
    const articleEl = container.querySelector('article');
    expect(articleEl).toBeInTheDocument();
  });

  test('chat messages have proper ARIA roles', () => {
    const { getByRole } = render(
      <ChatMessage message={mockMessage} />
    );

    const messageEl = getByRole('article');
    expect(messageEl).toHaveAttribute('aria-label', expect.stringContaining('message'));
  });

  test('loading states have aria-busy', () => {
    const { getByTestId } = render(<ArticleList isLoading={true} />);
    const container = getByTestId('article-list');
    expect(container).toHaveAttribute('aria-busy', 'true');
  });

  test('dialog traps focus correctly', async ({ page }) => {
    await page.goto('http://localhost:3000/dashboard');

    // Open modal
    await page.click('button:has-text("Create Workflow")');

    // Get all focusable elements
    const focusable = await page.locator(
      'button, a[href], input, select, textarea, [tabindex="0"]'
    ).all();

    // Focus should stay within modal
    const lastElement = focusable[focusable.length - 1];
    await lastElement.focus();
    await page.keyboard.press('Tab');

    const focused = await page.evaluate(
      () => document.activeElement.getAttribute('data-testid')
    );
    expect(focused).toContain('modal');
  });
});
```

Listing 13.3: Screen reader compatibility tests

Chapter 14

Test Maintenance and Evolution

14.1 Flaky Test Management

Flaky tests undermine confidence in the test suite. We manage them through:

1. **Detection**: GitHub Actions flags tests that pass/fail intermittently
2. **Quarantine**: Flaky tests are tagged with `@pytest.mark.flaky` and investigated
3. **Root Cause Analysis**:
   - Race conditions: Add explicit waits
   - Timing dependencies: Use mocked time or freezegun
   - External service calls: Ensure mocking is complete
   - Database state: Verify proper isolation between tests
4. **Resolution**: Fix underlying issue; remove quarantine once stable for 100 runs

14.2 Test Code Quality

Test code is maintained to the same standard as production code:

- Tests must be readable and maintainable
- DRY principle applied: use fixtures, factories, and helpers to avoid duplication
- Test naming must be clear: `test_<function>_<scenario>_<expected_result>`
- One assertion per test preferred; multiple related assertions acceptable
- Comments explaining complex test setup required
- Deprecated or redundant tests removed quarterly

14.3 Regression Test Suite

When bugs are fixed, regression tests are added to prevent recurrence:

```python
# Bug report: Articles with special characters in title failed to display
# Fix: Properly escape HTML in serializer
# Regression test added:

@pytest.mark.django_db
class TestRegressionSpecialCharacters:
    def test_article_with_ampersand_displays(self):
        article = ArticleFactory(title="AT&T Announces New Deal")
        response = client.get(f'/api/v1/articles/{article.id}/')

        assert response.status_code == 200
        data = response.json()
        assert data['title'] == "AT&T Announces New Deal"

    def test_article_with_quotes_displays(self):
        article = ArticleFactory(title='CEO Says "AI is Future"')
        response = client.get(f'/api/v1/articles/{article.id}/')

        assert response.status_code == 200
        data = response.json()
        assert data['title'] == 'CEO Says "AI is Future"'

    def test_article_with_html_tags_sanitized(self):
        article = ArticleFactory(
            title="<script>alert('xss')</script>Safe Title"
        )
        response = client.get(f'/api/v1/articles/{article.id}/')

        assert response.status_code == 200
        data = response.json()
        assert "<script>" not in data['title']
```

Listing 14.1: Regression test example

Chapter 15

Continuous Testing Culture

15.1 Testing Best Practices

1. **Test-First Mentality**: Write tests for critical logic before implementation
2. **Test Coverage Review**: Coverage metrics reviewed in code review
3. **Meaningful Test Names**: Test names should explain what is being tested and why
4. **Test Isolation**: Each test should run independently without side effects
5. **Explicit Over Implicit**: Use explicit assertions and clear test setup
6. **Fail Fast**: Tests should fail quickly on errors, not hang
7. **Comprehensive Mocking**: External dependencies must be mocked to keep tests fast
8. **Regular Cleanup**: Delete obsolete tests and refactor test code like production code

15.2 Team Training and Documentation

New team members receive:

1. Testing philosophy overview and pyramid concept
2. Tool tutorials: pytest, Jest, Playwright
3. Hands-on: Write a test for a small feature
4. Code review: Pair with experienced developer on test review
5. Documentation: Wiki pages on testing strategies and common patterns

15.3 Testing Tools and Libraries Summary

Category	**Tool**	**Purpose**
Backend Runner	pytest 7.x	Test execution
Backend ORM	pytest-django	Django integration
Backend Async	pytest-asyncio	AsyncIO testing
Backend Coverage	pytest-cov	Coverage measurement
Test Data	factory-boy	Test data factories
Fake Data	faker	Realistic data generation
HTTP Mocking	responses	Mock HTTP calls
Time Mocking	freezegun	Mock datetime
Frontend Runner	Jest 29.x	Test execution
Components	React Testing Library	Component testing
API Mocking	MSW	Network-level mocking
E2E Testing	Playwright	Browser automation
User Events	@testing-library/user-event	User interactions
Accessibility	jest-axe	A11y testing
Load Testing	Locust	Performance testing
Security	bandit	Static analysis
Security	OWASP ZAP	Dynamic scanning
Vulnerabilities	safety	Dependency check

Table 15.1: Summary of testing tools and libraries

Chapter 16

Conclusion

The SYNAPSE testing strategy outlined in this document provides comprehensive coverage across all layers of the platform. By following the testing pyramid, maintaining high code coverage, and consistently executing automated tests in the CI/CD pipeline, we ensure that:

- Every release is reliable and stable
- Bugs are caught early before reaching production
- Performance meets defined service-level objectives
- Security vulnerabilities are minimized
- User experience is consistent across features
- Technical debt is managed proactively

Success requires commitment to the testing culture outlined in this document. Testing is not a phase; it is integrated into the development workflow from day one. Developers write tests as they code, code reviews evaluate test quality, and the CI/CD pipeline enforces quality gates.

By maintaining this discipline and evolving our test suite alongside the product, SYNAPSE will remain a reliable, high-quality platform trusted by users to deliver accurate technology intelligence and insights.